Cloud Computing
Lab 4

Mariam Ibrahim
SE2

❒ Both Services are Working

```
mariamibrahim@mariams-air week4-lab % docker compose ps
NAME              IMAGE                        COMMAND           SERVICE           CREATED          STATUS                    PORTS
order-service     week4-lab-order-service      "python app.py"   order-service     17 seconds ago   Up 16 seconds (healthy)   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
product-service   week4-lab-product-service    "python app.py"   product-service   17 seconds ago   Up 16 seconds (healthy)   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
mariamibrahim@mariams-air week4-lab %
```

❒ Functional Testing

```
mariamibrahim@mariams-air week4-lab % curl http://localhost:5001/health
curl http://localhost:5001/products/1
curl http://localhost:5001/products/99
{"service":"product-service","status":"up"}
{"id":1,"name":"Laptop","price":1200}
{"error":"Product not found"}
mariamibrahim@mariams-air week4-lab %
```

```
mariamibrahim@mariams-air week4-lab % curl -X POST http://localhost:5002/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 2}'
{"message":"Order created","product":"Laptop","quantity":2,"total_price":2400}
mariamibrahim@mariams-air week4-lab %
```

❒ Failure Simulation

```
mariamibrahim@mariams-air week4-lab % docker stop product-service
product-service
mariamibrahim@mariams-air week4-lab % curl -X POST http://localhost:5002/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id": 2, "quantity": 1}'
{"error":"product-service unavailable"}
mariamibrahim@mariams-air week4-lab %
```

❒Restarting Product-service

```
mariamibrahim@mariams-air week4-lab % docker start product-service
docker compose ps
product-service
NAME              IMAGE                       COMMAND           SERVICE           CREATED         STATUS                                  PORTS
order-service     week4-lab-order-service     "python app.py"   order-service     6 minutes ago   Up 6 minutes (healthy)                  0.0.0.0:5002->5002/tcp, [
/tcp
product-service   week4-lab-product-service   "python app.py"   product-service   6 minutes ago   Up Less than a second (health: starting)   0.0.0.0:5001->5001/tcp, [
/tcp
mariamibrahim@mariams-air week4-lab %
```

```
mariamibrahim@mariams-air week4-lab % docker compose ps
NAME              IMAGE                       COMMAND           SERVICE           CREATED         STATUS                        PORTS
order-service     week4-lab-order-service     "python app.py"   order-service     8 minutes ago   Up 8 minutes (healthy)        0.0.0.0:5002->5002/tcp, [::]:5002->5002/tc
product-service   week4-lab-product-service   "python app.py"   product-service   8 minutes ago   Up About a minute (healthy)   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tc
mariamibrahim@mariams-air week4-lab %
```

Q1: Which parts show the benefits of microservices over a monolith?

Each service is independently deployable we stopped product-service without affecting order-service's ability to run. Each has its own Dockerfile, dependencies, and port. In a monolith, stopping one component means stopping everything.

Q2: Which new complexities were introduced by splitting into two services?

Network communication between services can fail we saw this when product-service went down and order-service had to retry and handle the 503. In a monolith, a function call never times out. We also need health checks, environment variables for service discovery, and Docker Compose just to run locally.

Q3: What would break if network latency increased or one service became slow?

order-service calls product-service synchronously with a 2-second timeout. If product-service becomes slow, every order request will hang for up to 2 seconds before timing out, making the entire order flow slow. This is called cascading latency one slow service degrades all services that depend on it.

Q4: Which 12-factor app principles are visible?

- **Config** — `PRODUCT_SERVICE_URL` is passed via environment variable, not hardcoded
- **Processes** — both services are stateless, storing no data locally
- **Port binding** — each service exposes its own port (5001, 5002)
- **Dependencies** — each service declares its own `requirements.txt`
- **Disposability** — `restart: unless-stopped` means services recover automatically